

[English](#)

APIJSON 通用文档

本文是通用文档，只和 APIJSON 协议有关，和 C#, Go, Java, JavaScript, PHP, Python, TypeScript 等开发语言无关。

具体开发语言相关的 配置、运行、部署 等文档见各个相关项目的文档，可以在首页点击对应语言的入口来查看。

<https://github.com/Tencent/APIJSON>

github.com/Tencent/APIJSON

README.md

APIJSON

零代码、热更新、全自动 ORM 库

🚀 后端接口和文档零代码，前端(客户端) 定制返回 JSON 的数据和结构

MySQL 5.7+ PostgreSQL 9.5+ SQLServer 2012+ Oracle 12C+ DB2 7.1+ TiDB 2.1+ ClickHouse 21.1+

Java 1.8+ Go 1.16+ CSharp 2.1+ PHP 7.0+ Node.js ES6+ Python 3+

Android 4.0+ iOS 7+ JavaScript ES6+

[English](#) [通用文档](#) [视频教程](#) [在线体验](#)

后端开发者可以先看 [图文入门教程1](#) 或 [图文入门教程2](#)（和本文档有出入的点以本文档为准。例如正则匹配 key? 已废弃，用 key~ 替代；例如 "@column":"store_id,sum(amt):totAmt" 中逗号，有误，应该用分号；隔开 SQL 函数，改为 "@column":"store_id;sum(amt):totAmt"）

- [1.示例](#)
- [2.对比传统方式](#)
 - [2.1 开发流程](#)
 - [2.2 前端请求](#)
 - [2.3 后端操作](#)
 - [2.4 前端解析](#)
 - [2.5 对应不同需求的请求](#)
 - [2.6 对应不同请求的结果](#)
- [3.设计规范](#)
 - [3.1 操作方法](#)
 - [3.2 功能符](#)

1.示例

获取用户

请求:

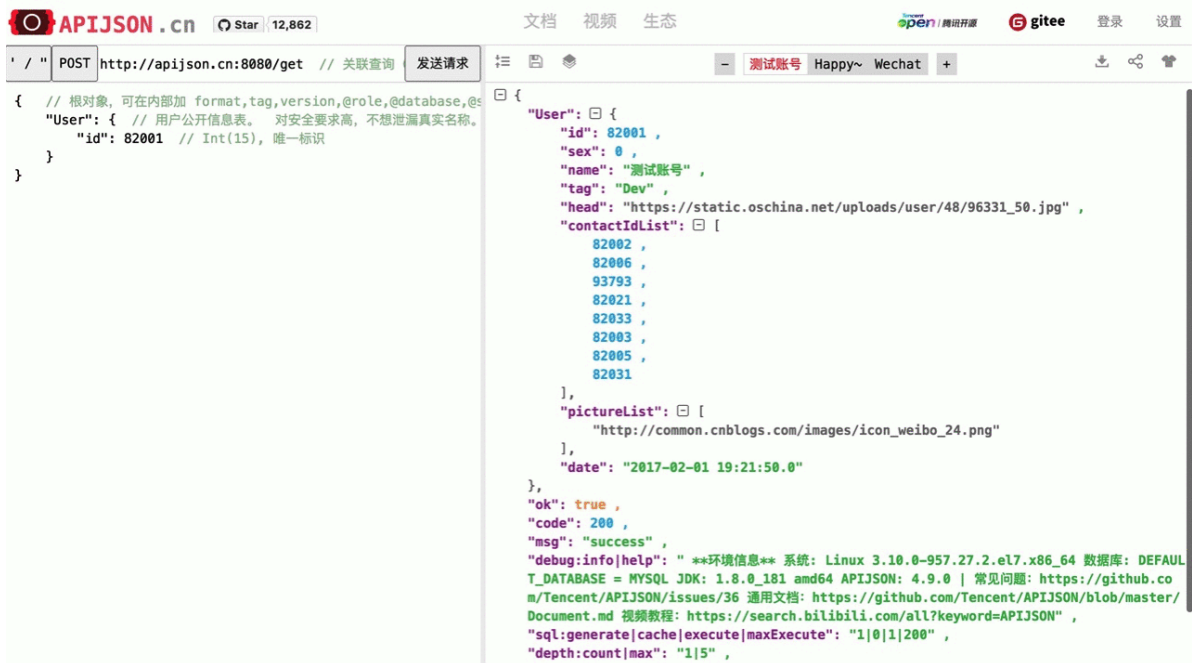
```
{
  "User": {
    "id": 38710
  }
}
```

[点击这里测试](#)

返回:

```
{
  "User": {
    "id": 38710,
    "sex": 0,
    "name": "TommyLemon",
    "tag": "Android&Java",
    "head": "http://static.oschina.net/uploads/user/1218/2437072_100.jpg?t=1461076033000",
    "date": 1485948110000,
    "pictureList": [
      "http://static.oschina.net/uploads/user/1218/2437072_100.jpg?t=1461076033000",
      "http://common.cnblogs.com/images/icon_weibo_24.png"
    ]
  },
  "code": 200,
  "msg": "success"
}
```

[GIF] APIJSON 各种单表对象查询: 简单查询、统计、分组、排序、聚合、比较、筛选字段、字段别名等



获取用户列表

请求:

```
{
  "[ ]":{
    "count":3, //只要3个
    "User":{
      "@column":"id,name" //只要id,name这两个字段
    }
  }
}
```

[点击这里测试](#)

返回:

```
{
  "[ ]": [
    {
      "User": {
        "id": 38710,
        "name": "TommyLemon"
      }
    },
    {
      "User": {
        "id": 70793,
        "name": "Strong"
      }
    },
    {
      "User": {
        "id": 82001,
        "name": "Android"
      }
    }
  ],
  "code": 200,
  "msg": "success"
}
```

[GIF] APIJSON 各种单表数组查询：简单查询、统计、分组、排序、聚合、分页、比较、搜索、正则、条件组合 等

The screenshot shows the APIJSON.cn website interface. The top navigation bar includes the APIJSON.cn logo, a star count of 12,862, and links for documentation, videos, and ecosystem. The main content area displays a REST client interface with a POST request to `http://apijson.cn:8080/get`. The request body is a JSON object with a `"Moment"` property. The response is a JSON object with a `"[ ]"` property containing an array of objects. The response is formatted with syntax highlighting and collapsible sections.

```
{
  // 根对象, 可在内部加 format,tag,version,@role,@database,@schema,@date
  "[ ]": { // 数组, 可在内部加 count,page,query,join 等关键词键值对
    "Moment": {} // 动态
  }
}
```

```
{
  "[ ]": [
    Object(2),
    {
      "Moment": Object(7)
    },
    {
      "Moment": {
        "id": 32,
        "userId": 82002,
        "date": "2017-02-08 16:06:11.0",
        "content": "tst",
        "praiseUserIdList": Array(3),
        "pictureList": Array(3)
      }
    },
    {
      "Moment": {
        "id": 58,
        "userId": 90814,
        "date": "2017-02-01 19:14:31.0",
        "content": "This is a Content...-435",
        "praiseUserIdList": [
          38710,
          82003,
          82005,
          93793,
          82006,
          82044
        ]
      }
    }
  ]
}
```

获取动态及发布者用户

请求:

```
{
  "Moment": {
  },
  "User": {
    "id@": "Moment/userId" //User.id = Moment.userId
  }
}
```

[点击这里测试](#)

返回:

```
{
  "Moment": {
    "id": 12,
    "userId": 70793,
    "date": "2017-02-08 16:06:11.0",
    "content": "1111534034"
  },
  "User": {
    "id": 70793,
    "sex": 0,
    "name": "strong",
    "tag": "djdj",
    "head": "http://static.oschina.net/uploads/user/585/1170143_50.jpg?t=1390226446000",
    "contactIdList": [
      38710,
      82002
    ],
    "date": "2017-02-01 19:21:50.0"
  },
  "code": 200,
  "msg": "success"
}
```

获取类似微信朋友圈的动态列表

请求:

```

{
  "[]":{                                     //请求一个数组
    "page":0,                               //数组条件
    "count":2,
    "Moment":{                             //请求一个名为Moment的对象
      "content$":"%a%"                     //对象条件，搜索content中包含a的动态
    },
    "User":{
      "id@":"/Moment/userId",              //User.id = Moment.userId 缺省引用赋值路径，从所处
容器的父容器路径开始
      "@column":"id,name,head"             //指定返回字段
    },
    "Comment[]":{                          //请求一个名为Comment的数组，并去除Comment包装
      "count":2,
      "Comment":{
        "momentId@":"/[]/Moment/id"        //Comment.momentId = Moment.id 完整引用赋值路径
      }
    }
  }
}

```

[点击这里测试](#)

返回：

```
{
  "[ ]": [
    {
      "Moment": {
        "id": 15,
        "userId": 70793,
        "date": 1486541171000,
        "content": "APIJSON is a JSON Transmission Structure Protocol...",
        "praiseUserIdList": [
          82055,
          82002,
          82001
        ],
        "pictureList": [
          "http://static.oschina.net/uploads/user/1218/2437072_100.jpg?t=1461076033000",
          "http://common.cnblogs.com/images/icon_weibo_24.png"
        ]
      },
      "User": {
        "id": 70793,
        "name": "Strong",
        "head": "http://static.oschina.net/uploads/user/585/1170143_50.jpg?t=1390226446000"
      },
      "Comment[ ]": [
        {
          "id": 176,
          "toId": 166,
          "userId": 38710,
          "momentId": 15,
          "date": 1490444883000,
          "content": "thank you"
        },
        {
          "id": 1490863469638,
          "toId": 0,
          "userId": 82002,
          "momentId": 15,
          "date": 1490863469000,
          "content": "Just do it"
        }
      ]
    },
    {
      "Moment": {
        "id": 58,
        "userId": 90814,
        "date": 1485947671000,
        "content": "This is a Content...-435",
```

```

    "praiseUserIdList":[
      38710,
      82003,
      82005,
      93793,
      82006,
      82044,
      82001
    ],
    "pictureList":[
      "http://static.oschina.net/uploads/img/201604/22172507_aMmH.jpg"
    ]
  },
  "User":{
    "id":90814,
    "name":7,
    "head":"http://static.oschina.net/uploads/user/51/102723_50.jpg?t=1449212504000"
  },
  "Comment[]":[
    {
      "id":13,
      "toId":0,
      "userId":82005,
      "momentId":58,
      "date":1485948050000,
      "content":"This is a Content...-13"
    },
    {
      "id":77,
      "toId":13,
      "userId":93793,
      "momentId":58,
      "date":1485948050000,
      "content":"This is a Content...-77"
    }
  ]
}
],
"code":200,
"msg":"success"
}

```

[\[GIF\] APIJSON 各种多表关联查询：一对一、一对多、多对一、各种条件 等](#)

APIJSON .cn Star 12,862 文档 视频 生态 open 腾讯开源 gitee 登录 设置

POST http://apijson.cn:8080/get // 关联查询 发送请求

```

{
  // 根对象, 可在内部加 format,tag,version,@role,@database,@s
  "Moment": { // 动态
    "id": 15 // Int(15), 唯一标识
  },
  "User": { // 用户公开信息表, 对安全要求高, 不想泄露真实名称。
    "id@": "Moment/userId" // 引用赋值, 从最外层开始的绝对(
  }
}

```

```

{
  "Moment": {
    "id": 15,
    "userId": 82002,
    "date": "2017-02-08 16:06:11.0",
    "content": "APIJSON is a JSON Transmission Structure Protocol...",
    "praiseUserIdList": [Array[9]],
    "pictureList": [
      "http://static.oschina.net/uploads/user/1218/2437072_100.jpg?t=1461076033000",
      "http://common.cnblogs.com/images/icon_weibo_24.png"
    ]
  },
  "User": {
    "id": 82002,
    "sex": 1,
    "name": "Happy~",
    "tag": "iOS",
    "head": "http://static.oschina.net/uploads/user/1174/2348263_50.png?t=1439773471000",
    "contactIdList": [
      82005,
      82001,
      38710
    ],
    "pictureList": [
    ],
    "date": "2017-02-01 19:21:50.0"
  },
  "ok": true,
}

```

[GIF] APIJSON 各种 JOIN: < LEFT JOIN, & INNER JOIN 等

APIJSON .cn Star 12,862 文档 视频 @explain 2/7 gitee 登录 设置

POST http://apijson.cn:8080/get // 关联查询 发送请求

```

{
  // 根对象, 可在内部加 format,tag,version,@role,@database,@s
  "[ ]": { // 数组, 可在内部加 count,page,query,join 等关键词
    "count": 3, // 每页数量
    "page": 2, // 分页页码
    "join": "</User/id@", // 多表连接: LEFT JOIN User, 同
    "Moment": { // 动态
      "@order": "date+" // 排序方式: +升序, -降序
    },
    "User": { // 用户公开信息表, 对安全要求高, 不想泄露真实名称。
      "id@": "/Moment/userId", // 引用赋值, 从对象父级开始
      "@column": "id,name,head" // 返回字段, 可传 字段(
    }
  },
  "@explain": true // 性能分析: true-开启 false-关闭, 返回执行
}

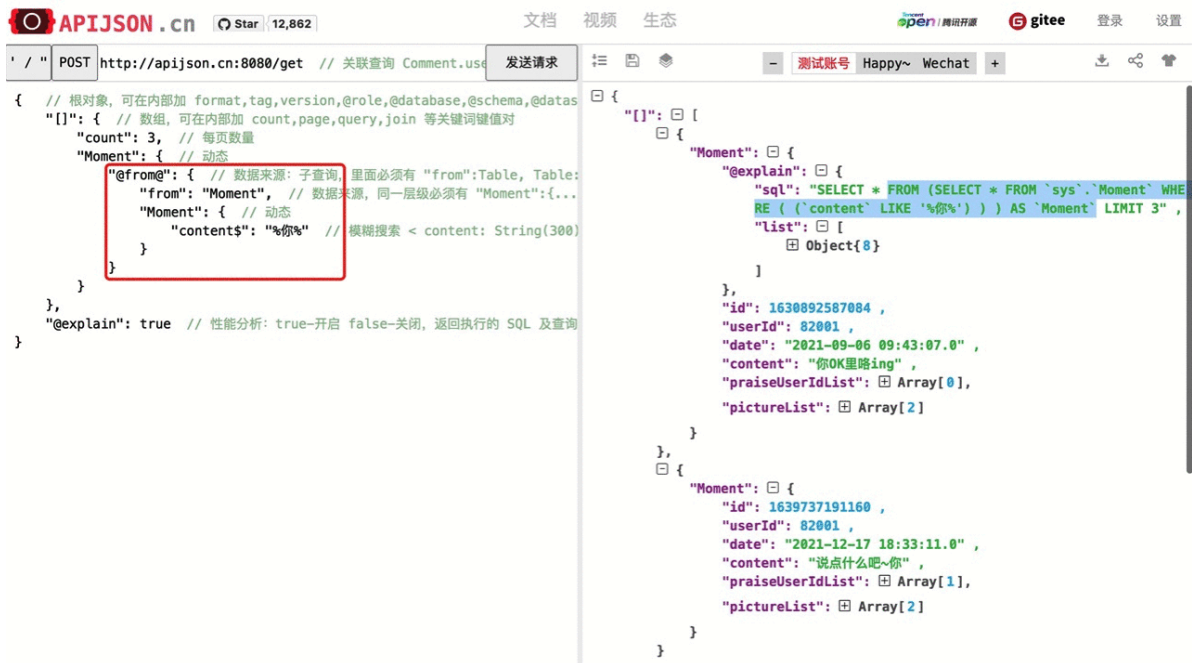
```

```

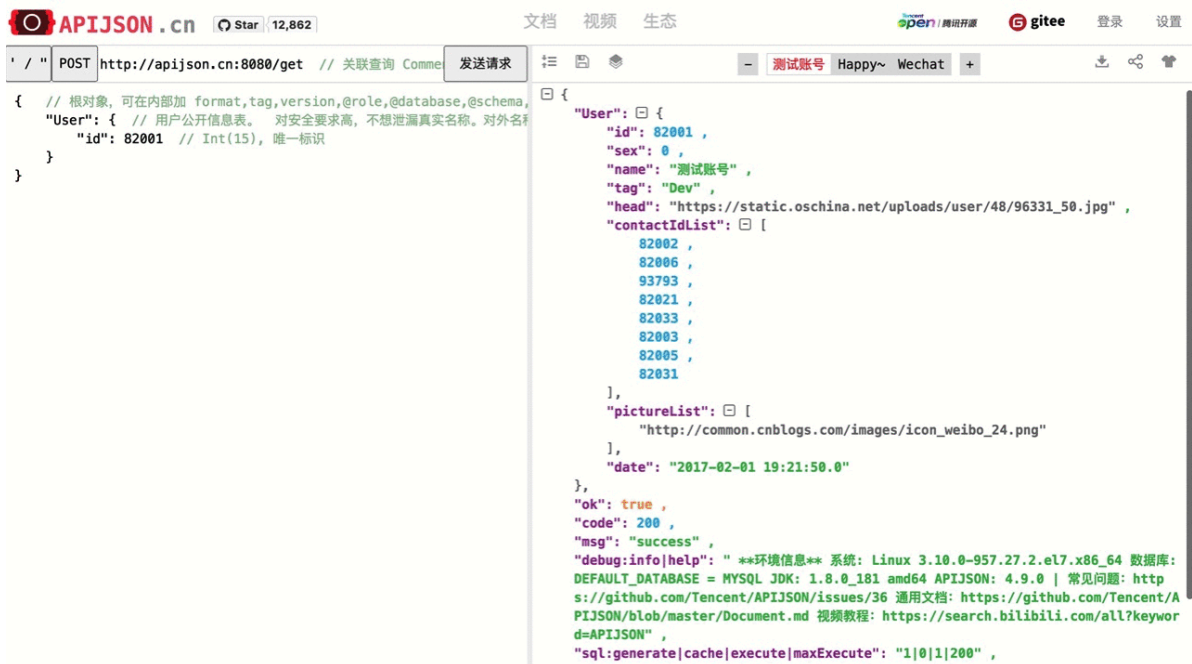
{
  "[ ]": [
    {
      "Moment": {
        "@explain": {
          "sql": "SELECT `Moment`.*, `User`.* FROM `sys`.`Moment` AS `Moment` LEFT JOIN ( SELECT `id`,`name`,`head` FROM `sys`.`apijson_user` ) AS `User` ON `User`.`id` = `Moment`.`userId` ORDER BY `Moment`.`date` ASC LIMIT 3 OFFSET 6",
          "list": [Array[2]]
        },
        "id": 15,
        "userId": 82002,
        "date": "2017-02-08 16:06:11.0",
        "content": "APIJSON is a JSON Transmission Structure Protocol...",
        "praiseUserIdList": [Array[9]],
        "pictureList": [Array[2]]
      },
      "User": {
        "@explain": {
          "sql": "SELECT `id`,`name`,`head` FROM `sys`.`apijson_user` WHERE ( `id` = 82002 ) LIMIT 1",
          "list": [
            {
              "id": 1,
              "select_type": "SIMPLE",
              "table": "apijson_user",
              "type": "const",
              "possible_keys": "PRIMARY",
              "key": "PRIMARY"
            }
          ]
        },
        "id": 82002,
        "sex": 1,
        "name": "Happy~",
        "tag": "iOS",
        "head": "http://static.oschina.net/uploads/user/1174/2348263_50.png?t=1439773471000",
        "contactIdList": [
          82005,
          82001,
          38710
        ],
        "pictureList": [
        ],
        "date": "2017-02-01 19:21:50.0"
      }
    }
  ]
}

```

[GIF] APIJSON 各种子查询: @from@ FROM, key@ =, key@>, key{}@ IN, key{}@ EXISTS 等



[GIF] APIJSON 部分功能演示集合, 由浅入深、由简单到复杂



[在线测试](#)

2.对比传统RESTful方式

2.1 开发流程

开发流程	传统方式	APIJSON
接口传输	等后端编辑接口，然后更新文档，前端再按照文档编辑请求和解析代码	前端按照自己的需求编辑请求和解析代码。 没有接口，更不需要文档！前端再也不用和后端沟通接口或文档问题了！
兼容旧版	后端增加新接口，用v2表示第2版接口，然后更新文档	什么都不需要做！

2.2 前端请求

前端请求	传统方式	APIJSON
要求	前端按照文档在对应URL后面拼接键值对	前端按照自己的需求在固定URL后拼接JSON
URL	不同的请求对应不同的URL，基本上有多少个不同的请求就得有多少个接口URL	相同的操作方法(增删改查)都用同一个URL， 大部分请求都用7个通用接口URL的其中一个
键值对	key=value	key:value

前端请求	传统方式	APIJSON
结构	同一个URL内table_name只能有一个 base_url/get/table_name? key0=value0&key1=value1...	同一个URL后TableName可传任意数量个 <pre>base_url/get/ { TableName0:{ key0:value0, key1:value1, ... }, TableName1:{ ... } ... }</pre>

2.3 后端操作

后端操作	传统方式	APIJSON
解析和返回	取出键值对，把键值对作为条件用预设的方式去查询数据库，最后封装JSON并返回给前端	把Parser#parse方法的返回值返回给前端就行
返回JSON结构的设定方式	由后端设定，前端不能修改	由前端设定，后端不能修改

2.4 前端解析

前端解析	传统方式	APIJSON
查看方式	查文档或问后端，或等请求成功后看日志	看请求就行，所求即所得，不用查、不用问、不用等。也可以等请求成功后看日志
解析方法	用JSON解析器来解析JSONObject	可以用JSONResponse解析JSONObject，或使用传统方式

2.5 前端对应不同需求的请求

前端的请求	传统方式	APIJSON
User	base_url/get/user?id=38710	<pre>base_url/get/ { "User":{ "id":38710 } }</pre>
Moment和 对应的 User	分两次请求 Moment: base_url/get/moment?userId=38710 User: base_url/get/user?id=38710	<pre>base_url/get/ { "Moment":{ "userId":38710 }, "User":{ "id":38710 } }</pre>
User列表	base_url/get/user/list? page=0&count=3&sex=0	<pre>base_url/get/ { "User[]":{ "page":0, "count":3, "User":{ "sex":0 } } }</pre>
Moment列表, 每个 Moment包 括 1.发布者 User 2.前3条 Comment	Moment里必须有 1.User对象 2.Comment数组 base_url/get/moment/list? page=0&count=3&commentCount=3	<pre>base_url/get/ { "[]":{ "page":0, "count":3, "Moment":{ "User":{ "id@":"/Moment/userId" }, "Comment[]":{ "count":3, "Comment":{ "momentId@":"/Moment/id" } } } } }</pre>

User发布前端的请求的Moment	1.Moment里必须有User对象和Comment数组 传统方式	APIJSON
列表， 每个Moment包括 1.发布者 User 2.前3条 Comment	2.字段名必须查接口文档，例如评论数量字段名可能是 commentCount,comment_count或者简写cmt_count等各种奇葩写法... base_url/get/moment/list? page=0&count=3 &commentCount=3&userId=38710	有以下几种方式: ① 把以上请求里的 "Moment":{,"User": { "id@":"/Moment/userId"} 改为 "Moment":{"userId":38710},"User":{"id":38710}. ② 或把User放在上面的最外层省去重复的User base_url/get/ { "User":{ "id":38710 }, "[]":{ "page":0, "count":3, "Moment":{ "userId":38710 }, "Comment[]":{ "count":3, "Comment":{ "momentId@":"/[]/Moment/id" } } } } ③ 如果User之前已经获取到了，还可以不传User来节省请求和返回数据的流量并提升速度 base_url/get/ { "[]":{ "page":0, "count":3, "Moment":{ "userId":38710 }, "Comment[]":{ "count":3, "Comment":{ "momentId@":"/[]/Moment/id" } } } }

前端的请求	传统方式	APIJSON
		<pre>_{ }</pre>

2.6 后端对应不同请求的返回结果

后端的返回结果	传统方式	APIJSON
User	<pre>{ "data":{ "id":38710, "name":"xxx", ... }, "code":200, "msg":"success" }</pre>	<pre>{ "User":{ "id":38710, "name":"xxx", ... }, "code":200, "msg":"success" }</pre>
Moment和对应的User	分别返回两次请求的结果，获取到Moment后取出userId作为User的id条件去查询User Moment: <pre>{ "data":{ "id":235, "content":"xxx", ... }, "code":200, "msg":"success" }</pre> User: <pre>{ "data":{ "id":38710, "name":"xxx", ... }, "code":200, "msg":"success" }</pre>	一次性返回，没有传统方式导致的 长时间等待结果、两次结果间关联、线程多次切换 等问题 <pre>{ "Moment":{ "id":235, "content":"xxx", ... }, "User":{ "id":38710, "name":"xxx", ... }, "code":200, "msg":"success" }</pre>

后端的返回结果	传统方式	APIJSON
User列表	<pre>{ "data":[{ "id":38710, "name":"xxx", ... }, { "id":82001, ... }, ...], "code":200, "msg":"success" }</pre>	<pre>{ "User[]":[{ "id":38710, "name":"xxx", ... }, { "id":82001, ... }, ...], "code":200, "msg":"success" }</pre>

后端的返回结果	传统方式	APIJSON
Moment列表，每个Moment包括发布者User和前3条Comment	Moment里必须有 1.User对象 2.Comment数组 <pre> { "data":[{ "id":235, "content":"xxx", ..., "User":{ ... }, "Comment":[...] }, { "id":301, "content":"xxx", ..., "User":{ ... }, ... }, ...], "code":200, "msg":"success" } </pre>	1.高灵活，可任意组合 2.低耦合，逻辑很清晰 <pre> { "[]":[{ "Moment":{ "id":235, "content":"xxx", ... }, "User":{ ... }, "Comment[]":[...] }, { "Moment":{ "id":301, "content":"xxx", ... }, "User":{ ... }, ... }, ...], "code":200, "msg":"success" } </pre>
User发布的Moment列表，每个Moment包括发布者User和前3条Comment	1.大量重复User，浪费流量和服务器性能 2.优化很繁琐，需要后端扩展接口、写好文档，前端/前端再配合优化 <pre> { "data":[{ "id":235, "content":"xxx", ..., "User":{ </pre>	以上不同请求方式的结果: ① 常规请求 <pre> { "[]":[{ "Moment":{ "id":235, "content":"xxx", ... }, "User":{ "id":38710, </pre>

后端的返回结果	传统方式	APIJSON
	<pre>"id":38710, "name":"Tommy" ... }, "Comment":[...] ... }, { "id":470, "content":"xxx", ..., "User":{ "id":38710, "name":"Tommy" ... }, "Comment":[...] ... }, { "id":511, "content":"xxx", ..., "User":{ "id":38710, "name":"Tommy" ... }, "Comment":[...] ... }, { "id":595, "content":"xxx", ..., "User":{ "id":38710, "name":"Tommy" ... }, "Comment":[...] }</pre>	<pre>"name":"Tommy" ... }, "Comment":[...] }, ...], "code":200, "msg":"success" } ② 省去重复的User { "User":{ "id":38710, "name":"Tommy", ... }, "[]":[{ "Moment":{ "id":235, "content":"xxx", ... }, "Comment":[...] }, ...], "code":200, "msg":"success" } ③ 不查询已获取到的User { "[]":[{ "Moment":{ "id":235, "content":"xxx", ... }, "Comment":[...] }, ...], }</pre>

后端的返回结果	传统方式	APIJSON
	<pre>] ... }, ...], "code":200, "msg":"success" }</pre>	<pre>], "code":200, "msg":"success" }</pre>

- 1.base_url指基地址，一般是顶级域名，其它分支url都是在base_url后扩展。如base_url:<http://apijson.cn:8080/>，对应的GET分支url:<http://apijson.cn:8080/get/>。下同。
- 2.请求中的key或value任意一个为null值时，这个 key:value键值对 被视为无效。下同。
- 3.请求中的 / 需要转义。JSONRequest.java已经用URLEncoder.encode转义，不需要再写；但如果是浏览器或Postman等直接输入url/request，需要把request中的所有 / 都改成 %252F 。下同。
- 4.code，指返回结果中的状态码，200表示成功，其它都是错误码，值全部都是HTTP标准状态码。下同。
- 5.msg，指返回结果中的状态信息，对成功结果或错误原因的详细说明。下同。
- 6.code和msg总是在返回结果的同一层级成对出现。对所有请求的返回结果都会在最外层有一对总结式code和msg。下同。
- 7.id等字段对应的值仅供说明，不一定是数据库里存在的，请求里用的是真实存在的值。下同。

3.设计规范

3.1 操作方法

方法及说明	URL	Request	Response
-------	-----	---------	----------

方法及说明	URL	Request	Response
GET: 普通获取数据, 可用浏览器调试	base_url/get/	<pre>{ TableName:{ ... } }</pre> {...}内为限制条件 例如获取一个 id = 235 的 Moment: <pre>{ "Moment":{ "id":235 } }</pre> 后端校验通过后自动解析为 SQL 并执行: <pre>SELECT * FROM Moment WHERE id=235 LIMIT 1</pre>	<pre>{ TableName:{ ... }, "code":200, "msg":"success" }</pre> 例如 <pre>{ "Moment":{ "id":235, "userId":38710, "content":"APIJSON,let interfaces and documents go to hell !" }, "code":200, "msg":"success" }</pre>
HEAD: 普通获取数量, 可用浏览器调试	base_url/head/	<pre>{ TableName:{ ... } }</pre> {...}内为限制条件 例如获取一个 id = 38710 的 User 所发布的 Moment 总数: <pre>{ "Moment":{ "userId":38710 } }</pre> 后端校验通过后自动解析为 SQL 并执行: <pre>SELECT count(*) FROM Moment WHERE userId=38710 LIMIT 1</pre>	<pre>{ TableName:{ "code":200, "msg":"success", "count":10 }, "code":200, "msg":"success" }</pre> 例如 <pre>{ "Moment":{ "code":200, "msg":"success", "count":10 }, "code":200, "msg":"success" }</pre>
GETS: 安全/私密获取数据, 用于获取钱包等 对安全性要求高的数据	base_url/gets/	最外层加一个 "tag":tag, 例如 "tag":"Privacy", 其它同GET	同GET
HEADS: 安全/私密获取数量, 用于获取银行卡数量等 对安全性要求高的数据总 数	base_url/heads/	最外层加一个 "tag":tag, 例如 "tag":"Verify", 其它同HEAD	同HEAD

方法及说明	URL	Request	Response
POST: 新增数据	base_url/post/	单个: <pre>{ TableName:{ ... }, "tag":tag }</pre> {...}中id由后端生成, 不能传 例如当前登录用户 38710 发布一个新 Comment: <pre>{ "Comment":{ "momentId":12, "content":"APIJSON,let interfaces and documents go to hell !" }, "tag":"Comment" }</pre> 后端校验通过后自动解析为 SQL 并执行: <pre>INSERT INTO Comment(userId,momentId,content) VALUES(38710,12,'APIJSON,let interfaces and documents go to hell !')</pre> 批量: <pre>{ TableName[]:{ ... },{ ... }, "tag":tag }</pre> {...}中id由后端生成, 不能传 例如当前登录用户 82001 发布 2 个 Comment: <pre>{ "Comment[]":[{ "momentId":12, "content":"APIJSON,let interfaces and documents go to hell !" },{ "momentId":15, "content":"APIJSON is a JSON transmision protocol." }], "tag":"Comment[]" }</pre> 后端校验通过后自动解析为 SQL 并执行: <pre>INSERT INTO Comment(userId,momentId,content) VALUES(82001,12,'APIJSON,let interfaces and documents go to hell !'); INSERT INTO Comment(userId,momentId,content) VALUES(82001,15,'APIJSON is a JSON transmision protocol.');</pre>	单个: <pre>{ TableName:{ "code":200, "msg":"success", "id":38710 }, "code":200, "msg":"success" }</pre> 例如 <pre>{ "Comment":{ "code":200, "msg":"success", "id":120 }, "code":200, "msg":"success" }</pre> 批量: <pre>{ TableName:{ "code":200, "msg":"success", "count":5, "id":[1, 2, 3, 4, 5] }, "code":200, "msg":"success" }</pre> 例如 <pre>{ "Comment":{ "code":200, "msg":"success", "count":2, "id":[1, 2] }, "code":200, "msg":"success" }</pre>

方法及说明	URL	Request	Response
PUT: 修改数据, 只修改所传字段	base_url/put/	<pre>{ TableName:{ "id":id, ... }, "tag":tag }</pre> {...} 中 id 或 id{} 至少传一个 例如当前登录用户 82001 修改 id = 235 的 Moment 的 content: <pre>{ "Moment":{ "id":235, "content":"APIJSON,let interfaces and documents go to hell !" }, "tag":"Moment" }</pre> 后端校验通过后自动解析为 SQL 并执行: <pre>UPDATE Moment SET content='APIJSON,let interfaces and documents go to hell !' WHERE id=235 AND userId=82001 LIMIT 1</pre> 批量除了 id:{} 也可类似批量 POST, 只是每个 {...} 里面都必须有 id。 "tag":"Comment[]" 对应对象 "Comment": {"id{}":[1,2,3]}, 表示指定记录全部统一设置; "tag":"Comment:[]" 多了冒号, 对应数组 "Comment[]":{"id":1},{"id":2},{"id":3}], 表示 每项单独设置	同POST
DELETE: 删除数据	base_url/delete/	<pre>{ TableName:{ "id":id }, "tag":tag }</pre> {...} 中 id 或 id{} 至少传一个, 一般只传 id 或 id{} 例如当前登录用户 82001 批量删除 id = 100,110,120 的 Comment: <pre>{ "Comment":{ "id{}":[100,110,120] }, "tag":"Comment[]" }</pre> 后端校验通过后自动解析为 SQL 并执行: <pre>DELETE FROM Comment WHERE id IN(100,110,120) AND userId=82001 LIMIT 3</pre>	<pre>{ TableName:{ "code":200, "msg":"success", "id[]":[100,110,120] "count":3 }, "code":200, "msg":"success" }</pre> 例如 <pre>{ "Comment":{ "code":200, "msg":"success", "id[]": [100,110,120], "count":3 }, "code":200, "msg":"success" }</pre>

方法及说明	URL	Request	Response
以上接口的简单形式: base_url/{method}/{tag}	GET: 普通获取数据 base_url/get/{tag}	例如安全/私密获取一个 id = 82001 的 Privacy: base_url/gets/Privacy/{id}:82001}	同以上对应的方法
	HEAD: 普通获取数量 base_url/head/{tag}	相当于 base_url/gets/{tag}:"Privacy", "Privacy":{"id":82001}}	
	GETS: 安全/私密获取数据 base_url/gets/{tag}	例如批量修改 id = 114, 124 的 Comment 的 content: base_url/put/Comemnt/{id}:[114,124], "content":"test multi put"	
	HEADS: 安全/私密获取数量 base_url/head/{tag}	{"id}:[114,124], "content":"test multi put"}	
	POST: 新增数据 base_url/post/{tag}	相当于 base_url/put/{tag}:"Comment", {"id}:[114,124], "content":"test multi put"}	
	PUT: 修改数据 base_url/put/{tag}	{"tag":"Comment", "Comment":{"id}:[114,124], "content":"test multi put"}	
	DELETE: 删除数据 base_url/delete/{tag}	_} }	

- 1.TableName指要查询的数据库表Table的名称字符串。第一个字符为大写字母，剩下的字符要符合英语字母、数字、下划线中的任何一种。对应的值的类型为JSONObject，结构是 {...}，里面放的是Table的字段(列名)。下同。
- 2."tag":tag 后面的tag是非GET、HEAD请求中匹配请求的JSON结构的标识，一般是要查询的Table的名称，由后端Request表中指定。下同。
- 3.GET、HEAD请求是开放请求，可任意组合任意嵌套。其它请求为受限制的安全/私密请求，对应的方法 (method)，标识 (tag)，版本 (version)，结构 (structure) 都必须和 后端Request表中所指定的 一一对应，否则请求将不被通过。version 不传、为 null 或 <=0 都会使用最高版本，传了其它有效值则会使用最接近它的最低版本。下同。
- 4.GETS与GET、HEADS与HEAD分别为同一类型的操作方法，请求稍有不同但返回结果相同。下同。
- 5.在HTTP通信中，自动化接口(get,gets,head,heads,post,put,delete) 全用HTTP POST请求。下同。
- 6.所有JSONObject都视为容器(或者文件夹)，结构为 {...}，里面可以放普通对象或子容器。下同。
- 7.每个对象都有一个唯一的路径(或者叫地址)，假设对象名为refKey，则用 key0/key1/.../refKey 表示。下同。

3.2 功能符

功能	键值对格式	使用示例
查询数组	"key[]":{}，后面是 JSONObject，key 可省略。当 key 和里面的 Table 名相同时，Table 会被提取出来，即 {Table:{Content}} 会被转化为 {Content}	{"User[]":{"User":{}}} ，查询一个 User 数组。这里 key 和 Table 名都是 User，User 会被提取出来，即 {"User":{"id", ...}} 会被转化为 {"id", ...}，如果要进一步提取 User 中的 id，可以把 User[] 改为 User-id[]，其中 - 用来分隔路径中涉及的 key

功能	键值对格式	使用示例
匹配选项范围	"key{}":[], 后面是 JSONArray, 作为 key 可取的值的选项	"id{}":["38710,82001,70793]", 对应 SQL 是 id IN(38710,82001,70793), 查询 id 符合 38710,82001,70793 中任意一个的一个 User 数组
匹配条件范围	"key{}"."条件0,条件1...", 条件为 SQL 表达式字符串, 可进行数字比较运算等	"id{}"."<=80000,>90000", 对应 SQL 是 id<=80000 OR id>90000, 查询 id 符合 id<=80000 id>90000 的一个 User 数组
包含选项范围	"key<>".value => "key<>".[value], key 对应值的类型必须为 JSONArray, value 值类型只能为 Boolean, Number, String 中的一种	"contactIdList<>":38710, 对应SQL是 json_contains(contactIdList,38710), 查询 contactIdList 包含 38710 的一个 User 数组
判断是否存在	"key){@":{ "from":"Table", "Table":{ ... } } 其中: { } 表示 EXISTS; key 用来标识是哪个判断; @ 后面是 子查询 对象, 具体见下方 子查询 的说明。	"id){@":{ "from":"Comment", "Comment":{ "momentId":15 } } WHERE EXISTS(SELECT * FROM Comment WHERE momentId=15)
远程调用函数	"key()":"函数表达式", 函数表达式为 function(key0,key1...), 会调用后端对应的函数 function(JSONObject request, String key0, String key1...), 实现 参数校验、数值计算、数据同步、消息推送、字段拼接、结构变换等特定的业务逻辑处理, 可使用 - 和 + 表示优先级, 解析 key-() > 解析当前对象 > 解析 key() > 解析子对象 > 解析 key+()	"isPraised()"."isContain(praiseUserIdList.userId)", 会调用远程函数 boolean isContain(JSONObject request, String array, String value), 然后变为 "isPraised":true 这种 (假设点赞用户 id 列表包含了 userId, 即这个 User 点了赞)
存储过程	"@key()":"SQL函数表达式", 函数表达式为 function(key0,key1...) 会调用后端数据库对应的存储过程 SQL 函数 function(String key0, String key1...) 除了参数会提前赋值, 其它和 远程函数 一致	"@limit":10, "@offset":0, "@procedure()":"getCommentByUserId(id,@limit,@offset)" 会转为 getCommentByUserId(38710,10,0) 来调用存储过程 SQL 函数 getCommentByUserId(IN id bigint, IN limit int, IN offset int) 然后变为 "procedure":{ "count":-1, "update":false, "list":[] } 其中 count 是指写操作影响记录行数, -1 表示不是写操作; update 是指是否为写操作 (增删改); list 为返回结果集
引用赋值	"key@":"key0/key1/.../refKey", 引用路径为用 / 分隔的字符串。以 / 开头的是缺省引用路径, 从声明 key 所处容器的父容器路径开始; 其它是完整引用路径, 从最外层开始。 被引用的 refKey 必须在声明 key 的上面。如果对 refKey 的容器指定了返回字段, 则被引用的 refKey 必须写在 @column 对应的值内, 例如 "@column":"refKey,key1,..."	"Moment":{ "userId":38710 } "User":{ "id@":"Moment/userId" } User 内的 id 引用了与 User 同级的 Moment 内的 userId, 即 User.id = Moment.userId, 请求完成后 "id@":"Moment/userId" 会变成 "id":38710
子查询	"key@":{ "range":"ALL", "from":"Table", // 可省略, 默认为首个表对象 key 名 "Table":{ ... } } 其中: range 可为 ALL,ANY; from 为目标表 Table 的名称; @ 后面的对象类似数组对象, 可使用 count 和 join 等功能。	"id@":{ "from":"Comment", // 可省略 "Comment":{ "@column":"min(userId)" } } WHERE id=(SELECT min(userId) FROM Comment)
模糊搜索	"key\$":"SQL搜索表达式" => "key\$":["SQL搜索表达式"], 任意 SQL 搜索表达式字符串, 如 %key%(包含 key), key%(以 key 开始), %k%e%y%(包含字母 k,e,y) 等, % 表示任意字符	"name\$":"%m%", 对应 SQL 是 name LIKE '%m%', 查询 name 包含 "m" 的一个 User 数组
正则匹配	"key~"."正则表达式" => "key-":["正则表达式"], 任意正则表达式字符串, 如 ^[0-9]+\$, *~ 忽略大小写, 可用于高级搜索	"name~"."^[0-9]+\$", 对应 SQL 是 name REGEXP '^[0-9]+\$', 查询 name 中字符全为数字的一个 User 数组
连续范围	"key%":["start,end"] => "key%":["start,end"]", 其中 start 和 end 都只能为 Number, String 中的一种, 如 "2017-01-01,2019-01-01", ["1,90000", "82001,100000"], 可用于连续范围内的筛选	"date%":["2017-10-01,2018-10-01]", 对应SQL是 date BETWEEN '2017-10-01' AND '2018-10-01', 查询在2017-10-01和2018-10-01期间注册的用户的一个 User数组
新建别名	"name:alias", name 映射为 alias, 用 alias 替代 name。可用于 column,Table,SQL 函数 等。只用于 GET 类型、HEAD 类型的请求	"@column":"told;parentId", 对应 SQL 是 toId AS parentId, 将查询的字段 told 变为 parentId 返回
增加或扩展	"key+":Object, Object的类型由key指定, 且类型为 Number,String,JSONArray 中的一种。如 82001,"apijson",["url0","url1"] 等。只用于 PUT 请求	"praiseUserIdList+":[82001], 对应 SQL 是 json_insert(praiseUserIdList,82001), 添加一个点赞用户 id, 即这个用户点了赞
减少或去除	"key-":Object, 与"key+"相反	"balance-":100.00, 对应SQL是 balance = balance - 100.00, 余额减少 100.00, 即花费了100元

功能	键值对格式	使用示例
比较运算	>, <, >=, <= 比较运算符, 用于 ① 提供 "id{}":"<=90000" 这种条件范围的简化写法 ② 实现子查询相关比较运算 不支持 "key":Object 和 "key!":Object 这两种写法, 直接用更简单的 "key":Object 和 "key!":Object 替代。	① "id<=":90000", 对应 SQL 是 <code>id<=90000</code>, 查询符合<code>id<=90000</code>的一个User数组 ② "id>@":{ "from":"Comment", "Comment":{ "@column":"min(userId)" }}" <code>WHERE id>(SELECT min(userId) FROM Comment)</code>
逻辑运算	&, , ! 逻辑运算符, 对应数据库 SQL 中的 AND, OR, NOT。 横或纵与: 同一键值对的值内条件默认 或连接, 可以在 key 后加逻辑运算符来具体指定; 不同键值对的条件默认 & 与连接, 可以用下面说明的对象关键词 @combine 来具体指定。 ① & 可用于 "key&{}":"条件"等 ② 可用于 "key {}":"条件", "key {}":[]等, 一般可省略 ③ ! 可单独使用, 如 "key!":Object, 也可像 &, 一样配合其他功能符使用 "key!":null 无效, null 值会导致整个键值对被忽略解析, 可以用 "key{}":"!=null" 替代, "key":null 同理, 用 "key{}":"=null" 替代。	① "id&{}":">80000,<=90000", 对应SQL是 <code>id>80000 AND id<=90000</code>, 即id满足<code>id>80000 & id<=90000</code> ② "id {}":">90000,<=80000", 同 <code>"id{}":">90000,<=80000"</code>, 对应 SQL 是 <code>id>90000 OR id<=80000</code>, 即 id 满足 <code>id>90000 id<=80000</code> ③ "id!{}":[82001,38710]", 对应 SQL 是 <code>id NOT IN(82001,38710)</code>, 即 id 满足 <code>!(id=82001 id=38710)</code>, 可过滤黑名单的消息

功能	键值对格式	使用示例
数组关键词，可自定义	"key":Object, key为 "[]" : {} 中 {} 内的关键词, Object 的类型由 key 指定 ① "count":5, 查询数量, 0 表示最大值, 默认值为 10, 默认最大值为 100 ② "page":1, 查询页码, 从 0 开始, 默认值为 0, 默认最大值为 100, 一般和 count 一起用 ③ "query":2, 查询内容 0-对象, 1-总数和分页详情, 2-数据、总数和分页详情 总数关键词为 total, 分页详情关键词为 info, 它们都和 query 同级, 通过引用赋值得到自定义 key:value 键值对, 不传则返回默认键值对, 例如 "total@":"/[]/total", "info@":"/[]/info" 这里query及total仅为GET类型的请求提供方便, 一般可直接用HEAD类型的请求获取总数 ④ "join":"&/Table0,</Table1/key1@" 或 "join":{ "&/Table0":{}, // 支持 ON 多个字段关联, "</Table1/key1@":{ // ON 只允许指定的 key1 关联 "key0":value0, // 其它ON条件 "key2":value2, ... "@combine":"...", // 其它ON条件的组合方式 "@column":"...", // 外层 SELECT "@group":"...", // 外层 GROUP BY "@having":"..." // 外层 HAVING } } 多表连接方式: "@ - APP JOIN "<" - LEFT JOIN ">" - RIGHT JOIN "&" - INNER JOIN " " - FULL JOIN " " - OUTER JOIN "*" - CROSS JOIN "^" - SIDE JOIN "(" - ANTI JOIN ")" - FOREIGN JOIN 其中 @ APP JOIN 为应用层连表, 会从已查出的主表里取得所有副表 key@ 关联的主表内的 refKey 作为一个数组 refKeys: [value0, value1...], 然后把原来副表 count 次查询 key=\$refKey 的 SQL 用 key IN(\$refKeys) 的方式合并为一条 SQL 来优化性能; 其它 JOIN 都是 SQL JOIN, 具体功能和 MySQL,PostgreSQL 等数据库的 JOIN 一一对应 "join":"</ViceTable/key@", "MainTable":{}, "ViceTable":{"key@":"/MainTable/refkey"} 会对应生成 MainTable LEFT JOIN ViceTable ON ViceTable.key=MainTable.refkey AND 其它ON条件 除了 = 等价关联, 也支持 ! 不等关联、> < >= <= 等比较关联和 \$ ~ {} <> 等其它复杂关联方式 ⑤ "otherKey":Object, 自定义关键词, 名称和以上系统关键词不一样, 且原样返回上传的值	① 查询User数组, 最多5个: "count":5 对应 SQL 是 LIMIT 5 ② 查询第3页的User数组, 每页5个: "count":5, "page":3 对应 SQL 是 LIMIT 5 OFFSET 15 ③ 查询User数组和对应的User总数: "[]":{ "query":2, "User":{} } "total@":"/[]/total", // 可省略 "info@":"/[]/info" // 可省略 返回的数据中, 总数及分页详情结构为: "total":139, // 总数 "info":{ // 分页详情 "total":139, // 总数 "count":5, // 每页数量 "page":0, // 当前页码 "max":27, // 最大页码 "more":true, // 是否还有更多 "first":true, // 是否为首页 "last":false // 是否为尾页 } ④ Moment INNER JOIN User LEFT JOIN Comment: "[]":{ "join":"&/User/id@,</Comment", "Moment":{ "@group":"id" // 主副表不是一对一, 要去除重复数据 }, "User":{ "name~":"t", "id@":"/Moment/userId" }, "Comment":{ "momentId@":"/Moment/id" } } ⑤ 每一层都加当前用户名: "User":{ "[]":{ "name@":"/User/name", // 自定义关键词 "Moment":{ } }

功能	键值对格式	使用示例
对象关键词，可自定义	"@key":Object, @key 为 Table:{} 中 {} 内的关键词, Object 的类型由 @key 指定 ① "@combine":"key0 (key1 & (key2 !key3))...", 条件组合方式, 最终按 (其它key条件 AND 连接) AND (key0条件 OR (key1条件 AND (key2条件 OR (NOT key3条件)))) 这种方式连接, 其中 "其它key" 是指与 @combine 在同一对象, 且未被它声明的条件 key, 默认都是 & 连接。注意不要缺少或多余任何一个空格。 ② "@column":"column;function(arg)..." , 返回字段 ③ "@order":"column0+column1-...", 排序方式 ④ "@group":"column0,column1...", 分组方式。如果 @column 里声明了 Table 的 id, 则 id 也必须在 @group 中声明; 其它情况下必须满足至少一个条件: 1.分组的 key 在 @column 里声明 2.Table 主键在 @group 中声明 ⑤ "@having":"function0(...)?value0;function1(...)?value1;function2(...)?value2..." // OR 连接, 或 "@having&":"function0(...)?value0;function1(...)?value1;function2(...)?value2..." // AND 连接, 或 "@having":{ "h0":"function0(...)?value0", "h1":"function1(...)?value1", "h2":"function2(...)?value2...", "@combine":"h0 & (h1 !h2)" // 任意组合, 非必传 } SQL 函数条件, 一般和 @group 一起用, 函数一般在 @column 里声明 ⑥ "@schema":"sys", 集合空间(数据库名/模式), 非默认的值可通过它来指定, 可以在最外层作为全局默认配置 ⑦ "@database":"POSTGRESQL", 数据库类型, 非默认的值可通过它来指定, 可以在最外层作为全局默认配置 ⑧ "@datasource":"DRUID", 跨数据源, 非默认的值可通过它来指定, 可以在最外层作为全局默认配置 ⑨ "@json":"key0,key1...", 转为 JSON 格式返回, 符合 JSONObject 则转为 {...}, 符合 JSONArray 则转为 [...] ⑩ "@role":"OWNER", 来访角色, 包括 UNKNOWN,LOGIN,CONTACT,CIRCLE,OWNER,ADMIN, 可以在最外层作为全局默认配置, 可自定义其它角色并重写 Verifier.verify 等相关方法来自定义校验 ⑪ "@explain":true, 性能分析, 可以在最外层作为全局默认配置 ⑫ "@raw":"key0,key1...", 其中 key0, key1 都对应键值对 "key0":"SQL片段或SQL片段的别名", "key1":"SQL片段或SQL片段的别名" 自定义原始SQL片段, 可扩展嵌套SQL函数等复杂语句, 必须是后端已配置的, 只有其它功能符都做不到才考虑, 谨慎使用, 注意防 SQL 注入 ⑬ "@otherKey":Object, 自定义关键词, 名称和以上系统关键词不一样, 且原样返回上传的值	① 搜索 name 或 tag 任何一个字段包含字符 a 的 User 列表: "name~":"a", "tag~":"a", "@combine":"name~ tag~" 对应SQL是 <code>name REGEXP 'a' OR tag REGEXP 'a'</code> ② 只查询 id,sex,name 这几列并且请求结果也按照这个顺序: "@column":"id,sex,name" 对应 SQL 是 <code>SELECT id,sex,name</code> ③ 查询按 name 降序、id 默认顺序 排序的 User 数组: "@order":"name-id" 对应 SQL 是 <code>ORDER BY name DESC,id</code> ④ 查询按 userId 分组的 Moment 数组: "@group":"userId,id" 对应 SQL 是 <code>GROUP BY userId,id</code> ⑤ 查询 按 userId 分组、id 最大值>=100 的 Moment 数组: "@column":"userId,max(id)", "@group":"userId" "@having":"max(id)>=100" 对应 SQL 是 <code>SELECT userId,max(id) ... GROUP BY userId HAVING max(id)>=100</code> 还可以指定函数返回名: "@column":"userId,max(id):maxId", "@group":"userId", "@having":"(maxId)>=100" 对应 SQL 是 <code>SELECT userId,max(id) AS maxId ... GROUP BY userId HAVING (maxId)>=100</code> ⑥ 查询 sys 内的 User 表: "@schema":"sys" 对应 SQL 是 <code>FROM sys.User</code> ⑦ 查询 PostgreSQL 数据库的 User 表: "@database":"POSTGRESQL" ⑧ 使用 Druid 连接池查询 User 表: "@datasource":"DRUID" ⑨ 将 VARCHAR 字符串字段 get 转为 JSONArray 返回: "@json":"get" ⑩ 查询当前用户的动态: "@role":"OWNER" ⑪ 开启性能分析: "@explain":true 对应 SQL 是 <code>EXPLAIN</code> ⑫ 统计最近一周偶数 userId 的数量 "@column":"date:left(date,10):day,sum(if(userId%2=0,1,0))", "@group":"day", "@having":"to_days(now())-to_days(`date`)<=7", "@raw":"@column,@having_","@group":"day","@having":"to_days(now())-to_days(`date`)<=7","@raw":"@column,@having_}}} 对应 SQL 是 <code>SELECT date, left(date,10) AS day, sum(if(userId%2=0,1,0)) ... GROUP BY day HAVING to_days(now())-to_days(`date`)<=7</code> ⑬ 从pictureList 获取第 0 张图片: "@position":0, // 自定义关键词 "firstPicture()":"getFromArray(pictureList.@position)"

功能	键值对格式	使用示例
全局 关键词	为最外层对象 {} 内的关键词。其中 @database, @schema, @datasource, @role, @explain 基本同对象关键词，见上方说明，区别是全局关键词会每个表对象中没有时自动放入，作为默认值。 ① "tag": "Table", 后面的 tag 是非 GET、HEAD 请求中匹配请求的 JSON 结构的标识，一般是要查询的 Table 的名称或该名称对应的数组 Table[] 或 Table[], 由后端 Request 表中指定。 ② "version": 1, 接口版本, version 不传、为 null 或 <=0 都会使用最高版本, 传了其它有效值则会使用最接近它的最低版本, 由后端 Request 表中指定。 ③ "format": true, 格式化返回 Response JSON 的 key, 一般是将 TableName 转为 tableName, TableName[] 转为 tableNameList, Table:alias 转为 alias, TableName-key[] 转为 tableNameKeyList 等小驼峰格式。	① 查隐私信息： {"tag": "Privacy", "Privacy": {"id": 82001}} ② 使用第 1 版接口查隐私信息： {"version": 1, "tag": "Privacy", "Privacy": {"id": 82001}} ③ 格式化朋友圈接口返回 JSON 中的 key： {"format": true, "[]": {"page": 0, "count": 3, "Moment": {"User": {"id@": "/Moment/userId"}, "Comment[]": {"count": 3, "Comment": {"momentId@": "[]/Moment/id"}}, "momentId@": "[]/Moment/id"}}, "momentId@": "[]/Moment/id"}}